

Experiment 13: To Implement Min Max Algorithm For Gaming

Aim

To write a Python program to implement the Minimax algorithm for gaming and find the best possible move for a player.

Objective

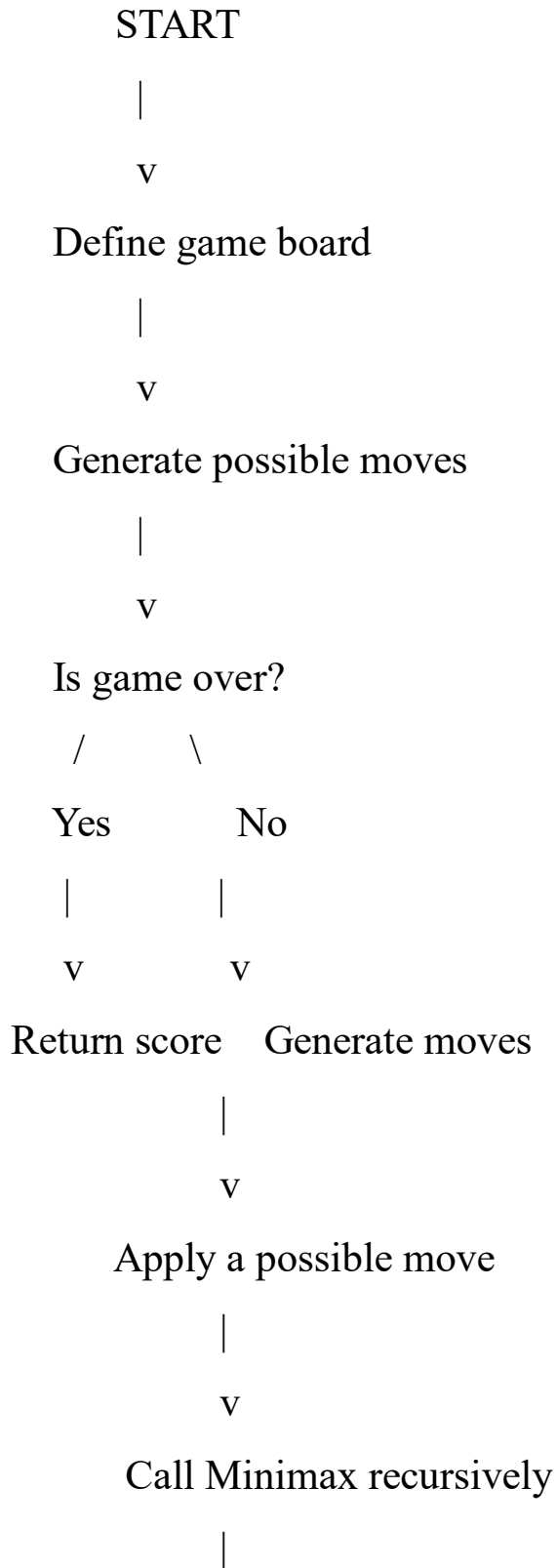
- To understand the concept of game playing in Artificial Intelligence.
- To understand the Minimax decision-making algorithm.
- To implement Minimax for a simple game.
- To select the best move for the AI player.
- To understand how the AI chooses a winning or optimal move.

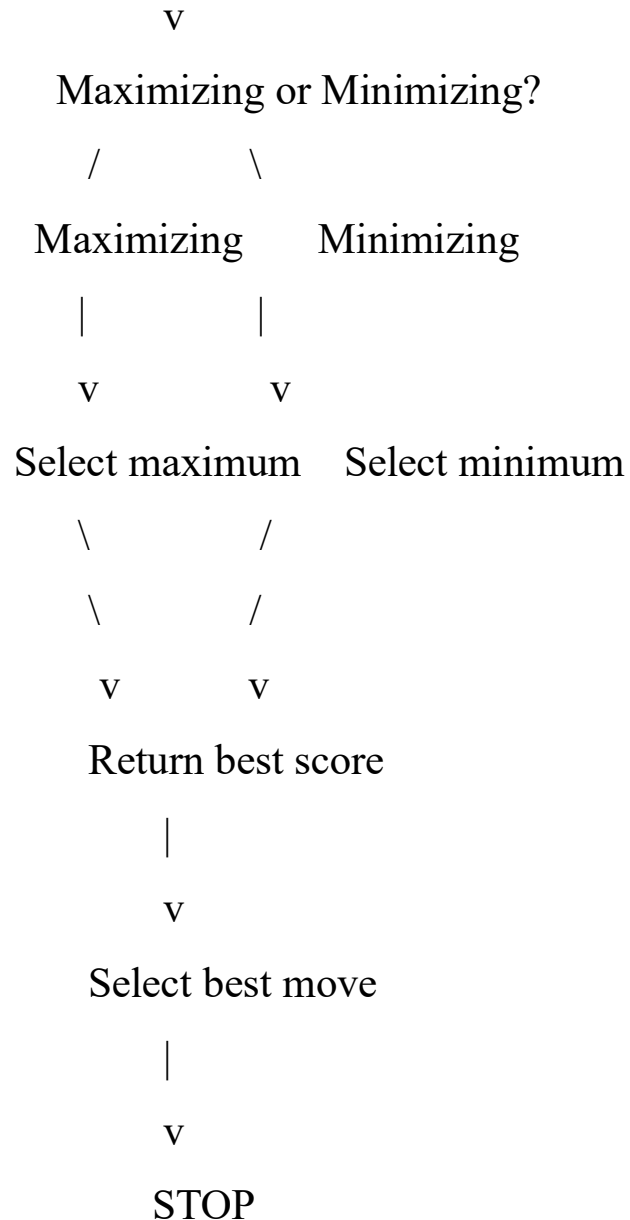
Algorithm

1. Start.
2. Define the game states and possible moves.
3. Check whether the current state is a terminal state.
4. If the AI wins, return a positive score.
5. If the opponent wins, return a negative score.
6. If the game is a draw, return zero.
7. Generate all possible moves.
8. For the maximizing player, select the move with the maximum score.
9. For the minimizing player, select the move with the minimum score.
10. Continue recursively until a terminal state is reached.
11. Select the move with the best Minimax score.

12. Display the best move.
13. Stop.

Flowchart





Code (Python)

Minimax Algorithm for Gaming

Tic Tac Toe using Minimax

```
board = [' ' for i in range(9)]
```

```
def display():
```

```
    print(board[0], '|', board[1], '|', board[2])
```

```
print('--+---+--')
print(board[3], '|', board[4], '|', board[5])
print('--+---+--')
print(board[6], '|', board[7], '|', board[8])
print()
```

```
def check_winner(player):
```

```
    combinations = [
```

```
        (0, 1, 2),
```

```
        (3, 4, 5),
```

```
        (6, 7, 8),
```

```
        (0, 3, 6),
```

```
        (1, 4, 7),
```

```
        (2, 5, 8),
```

```
        (0, 4, 8),
```

```
        (2, 4, 6)
```

```
    ]
```

```
    for a, b, c in combinations:
```

```
        if board[a] == board[b] == board[c] == player:
```

```
            return True
```

```
    return False
```

```
def minimax(is_maximizing):
```

```
# AI wins
```

```
if check_winner('O'):
```

```
    return 1
```

```
# Human wins
```

```
if check_winner('X'):
```

```
    return -1
```

```
# Draw
```

```
if ' ' not in board:
```

```
    return 0
```

```
if is_maximizing:
```

```
    best_score = -1000
```

```
for i in range(9):
```

```
    if board[i] == ' ':
```

```
        board[i] = 'O'
```

```
        score = minimax(False)
```

```
        board[i] = ' '
```

```
        best_score = max(best_score, score)
```

```
return best_score
```

```
else:
    best_score = 1000

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(True)
            board[i] = ' '
            best_score = min(best_score, score)

    return best_score
```

```
def best_move():

    best_score = -1000
    move = -1

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(False)
            board[i] = ' '

            if score > best_score:
                best_score = score
```

```
move = i
```

```
return move
```

```
# Game starts
```

```
while True:
```

```
    display()
```

```
    # Human move
```

```
    position = int(input("Enter your position (1-9): ")) - 1
```

```
    if position < 0 or position > 8 or board[position] != '':
```

```
        print("Invalid move!")
```

```
        continue
```

```
    board[position] = 'X'
```

```
    if check_winner('X'):
```

```
        display()
```

```
        print("You win!")
```

```
        break
```

```
    if ' ' not in board:
```

```
display()
print("Game Draw!")
break
```

```
# AI move
```

```
move = best_move()
```

```
board[move] = 'O'
```

```
print("AI selected position:", move + 1)
```

```
if check_winner('O'):
```

```
    display()
```

```
    print("AI wins!")
```

```
    break
```

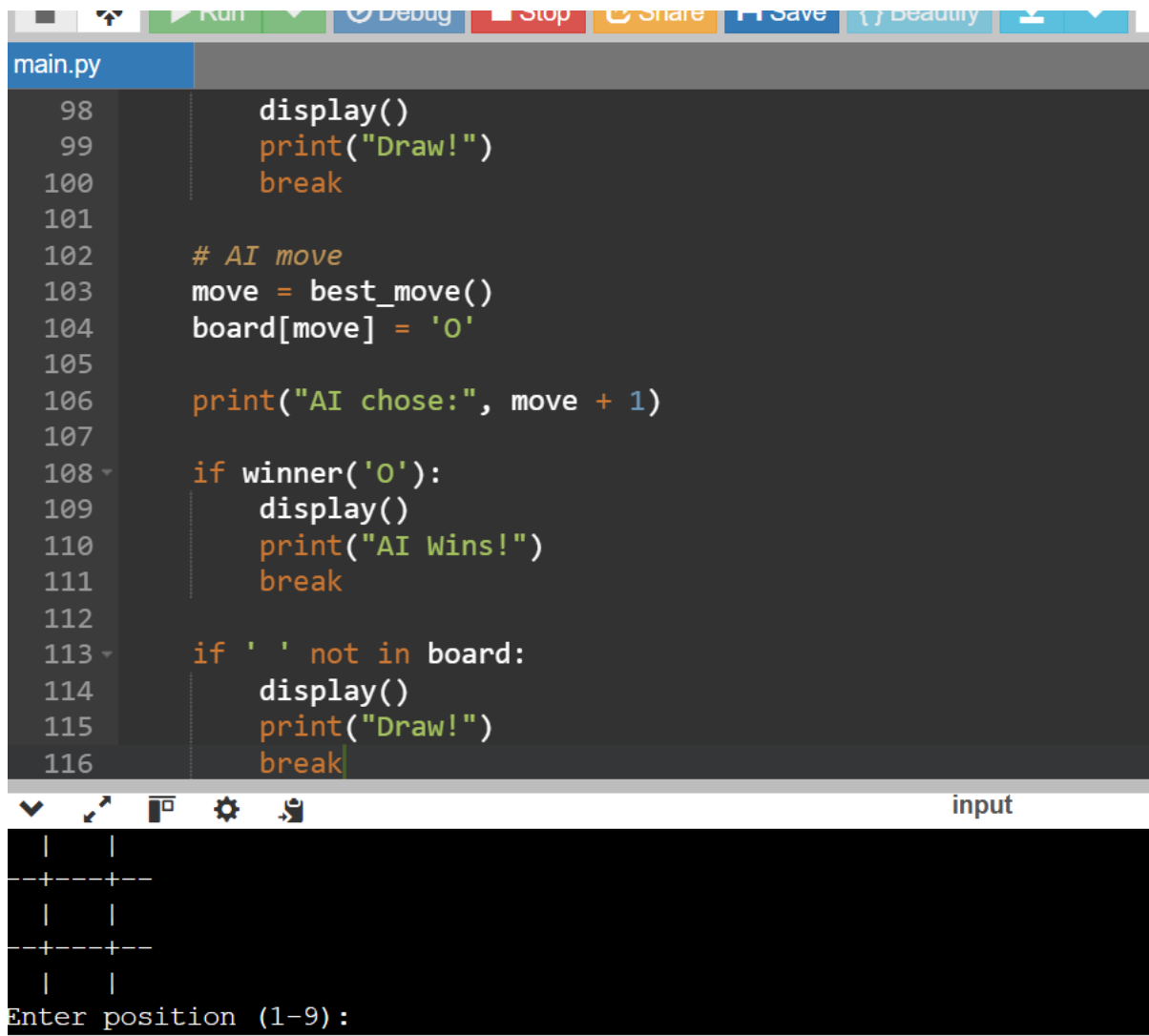
```
if ' ' not in board:
```

```
    display()
```

```
    print("Game Draw!")
```

```
    break
```

Output



```
main.py
98     display()
99     print("Draw!")
100    break
101
102    # AI move
103    move = best_move()
104    board[move] = 'O'
105
106    print("AI chose:", move + 1)
107
108    if winner('O'):
109        display()
110        print("AI Wins!")
111        break
112
113    if ' ' not in board:
114        display()
115        print("Draw!")
116        break
```

input

```
|  |
+--+
|  |
+--+
|  |
Enter position (1-9):
```

The AI evaluates the possible moves using the Minimax algorithm and chooses the move with the best score.

Result

Thus, the Python program to implement the Minimax algorithm for gaming was executed successfully, and the AI player selected the optimal moves.

Conclusion

The experiment helped in understanding the Minimax algorithm used in Artificial Intelligence game playing. The algorithm evaluates possible future moves and chooses the best move for the maximizing player while assuming that the opponent will choose the best move for the minimizing player.

